

Project Methodology: Medical RAG Pipeline

This document outlines the step-by-step methodology for building and querying a medical information retrieval system, as derived from the project notebooks.

Phase 1: Data Preparation & Indexing (Offline Pipeline)

This phase focuses on processing the raw data and building the searchable knowledge base.

Step 1: Data Loading and Enrichment (`metaData_preparation.ipynb`)

1. **Load Raw Data:** The process begins by loading the initial dataset, which consists of unstructured medical question-and-answer (Q&A) pairs from a `data.json` file.
2. **Parse Q&A:** Each record is parsed to cleanly separate the "question" and "answer" components.
3. **Metadata Enrichment:** A generative AI model (`gemini-2.5-flash`) is used to analyze each Q&A pair. For each entry, it generates structured metadata:
 - `category` : A low-level topic (e.g., "viral infection").
 - `entities` : A list of key medical entities (e.g., diseases, symptoms, drugs).
 - `keywords` : A list of short, relevant tokens for search.
 - `safety_level` : A risk assessment (low, medium, high).
4. **Consolidate Text:** The "question" and "answer" are combined into a single `combined_text` field. This text will be used for embedding.
5. **Save Enriched Data:** The newly structured and enriched dataset is saved as `enriched_records.csv`.

Step 2: Data Cleaning and Finalization (`data_analyst.ipynb`)

1. **Load Enriched Data:** The `enriched_records.csv` file from Step 1 is loaded.
2. **Filter Errors:** The dataset is filtered to remove any rows that produced errors during the enrichment process (e.g., API quota limits). The clean data (up to ID 9790) is retained.
3. **Prune Columns:** Unnecessary columns like the original `question`, `answer`, and `error` fields are dropped to create a final, clean dataset.
4. **Save Final Dataset:** The cleaned, finalized data is saved as `data.json`, which serves as the definitive source for the next phase.

Step 3: Vector Embedding and Indexing (`embedding_and_pinecone.ipynb`)

1. **Load Final Data:** The `data.json` file from Step 2 is loaded.
2. **Initialize Vector DB:** A connection is established with the Pinecone vector database. A new index (e.g., `medical-rag-index`) is created if one doesn't already exist, configured with the correct dimensions (3072) for the embedding model.
3. **Generate Embeddings (in batches):** The system iterates through the dataset in batches (e.g., 100 records at a time).

- For each batch, the `combined_text` of every record is fed into the `models/gemini-embedding-001` embedding model.
 - This model converts each text chunk into a 3072-dimension numerical vector (an "embedding"). This is done using `task_type="RETRIEVAL_DOCUMENT"`.
4. **Upsert to Pinecone:** The generated embeddings are "upserted" (uploaded) into the Pinecone index. Each vector is stored along with its corresponding metadata (ID, category, keywords, and the `combined_text` itself).
 5. **Verification:** The process concludes by checking the Pinecone index statistics to confirm that all 9,790 vectors have been successfully indexed.

Phase 2: Query Retrieval (Online Pipeline)

This phase describes what happens when a user actively searches for information.

Step 4: Real-time Query Processing (`query_retrieve.ipynb`)

1. **Receive User Query:** The system receives a natural language question from a user (e.g., "I have red pimples on my face, how dangerous is it?").
2. **Embed User Query:** The same embedding model (`models/gemini-embedding-001`) is used to convert the user's query into a 3072-dimension vector.
 - **Crucially**, this uses `task_type="RETRIEVAL_QUERY"` to optimize the embedding for searching.
3. **Search Vector DB:** The user's query vector is used to search the Pinecone index. Pinecone performs a similarity search (using "cosine" metric) to find the vectors in the index that are most mathematically similar to the query vector.
4. **Retrieve Context:** The system retrieves the `top_k` (e.g., top 5) most relevant matches. From these matches, it extracts the original `combined_text` (which was stored as metadata).
5. **Return Context:** This list of relevant text chunks is returned as the "retrieved context." This context can then be used by a separate generative model to formulate a final, accurate answer.